

ABSIA

Autonomous Bayesian System Intelligence Agent

Real-World Testing & Benchmark Plan

Phase-by-phase execution guide for AI agent — with strict rules, pass/fail criteria, remediation prompts, and buyer-ready evidence collection.

Repository	github.com/kuldeep-poonia/Real-time-causal-inference-engine
Docker Hub	poonia98/absia:latest
Language	Go 1.23+ (zero external dependencies)
Binary Size	~8 MB (scratch base)
Version	2.1.0
Author	Kuldeep Poonia
Document	v1.0 — June 2026

SECTION 0 — STRICT AI AGENT OPERATING RULES

■ These rules are ABSOLUTE. The AI agent executing this plan must follow them without exception. Any deviation invalidates the benchmark results.

RULE 1 — Deep Analysis Before Action

Before executing ANY phase, the agent MUST clone the repository, read every file in every package (pkg/, internal/, deploy/, tests/, .github/), and build a complete mental model: call graph, data flow, dependency graph, and API contract. No phase execution is permitted until this analysis is complete and documented.

RULE 2 — Phase Gating Is Non-Negotiable

Phases are sequential. Phase N+1 cannot start until Phase N has produced ALL required evidence files and every pass criterion has been met with actual measured values — not estimated or assumed values. No skipping. No parallel execution.

RULE 3 — Evidence Must Be Raw and Unmodified

All benchmark numbers, API responses, docker stats output, and logs must be captured as-is and stored in /evidence/phaseN/ directories. The agent must NOT round, approximate, or editorialize raw evidence. Numbers go in as measured.

RULE 4 — On Any Test Failure, STOP and Diagnose

If a test or criterion fails, the agent must immediately stop, run the Failure Diagnosis Prompt for that phase, document the root cause, propose a fix, and wait for human approval before continuing. The agent must NOT auto-patch and retry.

RULE 5 — No Synthetic or Mocked Data in External Test Phases

From Phase 3 onwards (External Stack Testing), all metrics must come from real running containers. The agent must NOT inject synthetic data via /ingest to inflate confidence scores. Only /ingest with real measured values is permitted.

RULE 6 — Safety Gate Must Never Be Bypassed

The agent must never modify ABSIA source code to lower thresholds, disable the safety gate, or increase confidence scores. The product is tested as shipped.

RULE 7 — Competitor Comparison Must Be Factual

When documenting Prometheus/Grafana comparison, the agent captures real screenshots and real alert counts. It must not describe competitors as 'worse' — only document what each tool returned for the same failure scenario.

RULE 8 — All Prompts in This Document Are Exact Instructions

Every 'AI Agent Prompt' box in this document is an exact instruction the agent must execute verbatim when triggered. The agent must not paraphrase or abbreviate.

■ AI AGENT PROMPT — PRE-EXECUTION — MANDATORY REPOSITORY ANALYSIS

You are about to execute the ABSIA testing plan. Before doing ANYTHING else, perform the following complete analysis and output it as a structured report:

1. Clone the repository: `git clone https://github.com/kuldeep-poonia/Real-time-causal-inference-engine`
 2. For EVERY file in EVERY directory (`main.go`, `pkg/*`, `internal/*`, `deploy/*`, `tests/*`, `.github/workflows/*`), read the complete file content.
 3. Build and output the following artifacts:
 - a) PACKAGE DEPENDENCY GRAPH — which pkg imports which, draw as ASCII DAG
 - b) CALL GRAPH — `main()` -> which functions -> which sub-functions for `/analyze` endpoint
 - c) DATA FLOW MAP — how a metric enters `/ingest`, flows through 5 phases, exits `/analyze`
 - d) API CONTRACT TABLE — every endpoint, method, request schema, response schema, auth
 - e) IDENTIFIED RISKS — any race conditions, missing error handling, hardcoded values, global state, or production risks you observe in the code
 4. Output a READINESS SCORE (0-100) with justification before starting Phase 1.
- DO NOT begin Phase 1 until this analysis report is complete.



PHASE 1 — Local Baseline Verification

Confirm the binary builds, runs, and responds correctly on a clean machine

Objective

Prove that ABSIA starts cleanly from the Docker Hub image on a machine with only Docker installed, discovers containers automatically, accepts manual ingest, and returns correct API responses. This is the foundation — if Phase 1 fails, nothing else is valid.

1.1 Environment Setup

```
# Pull the image fresh – no local builds allowed in Phase 1
docker pull poonia98/absia:latest

# Start a minimal 3-container test stack
cat > /tmp/phase1-compose.yml <<'EOF'
services:
  absia:
    image: poonia98/absia:latest
    ports: ['8080:8080']
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
  api-service:
    image: nginx:alpine
  worker-service:
    image: redis:7-alpine
EOF
docker compose -f /tmp/phase1-compose.yml up -d
sleep 10 # Wait for ABSIA to start
```

1.2 Test Sequence

Test ID	Test Name	Action	Expected Result
T1.1	Health Check	GET /health returns {status:ok, ready:true}	HTTP 200, ready=true
T1.2	Container Discovery	GET /nodes after 60s shows nginx and redis containers	node_count >= 2
T1.3	Pipeline Readiness	GET /nodes — both containers show pipeline_ready=true after 4 samples	pipeline_ready=true for all nodes
T1.4	Manual Ingest	POST /ingest with test node data, verify HTTP 204	HTTP 204 response
T1.5	Analyze Endpoint	POST /analyze after 4 ingested samples	execution_time_ms < 200, success=true
T1.6	Explain Endpoint	POST /explain, verify causal chain present	causes[] not empty, uncertainty values present
T1.7	Safety Gate (insufficient data)	POST /act on node with < 4 samples	fallback_triggered=true, HTTP 503
T1.8	Prometheus Metrics	GET /metrics — verify ABSIA metrics present	absia_nodes_total > 0
T1.9	Memory Baseline	docker stats absia after 5 min steady state	MEM < 80MB

Test ID	Test Name	Action	Expected Result
T1.10	Image Size	docker images poonia98/absia	SIZE < 15MB

■ PHASE PASS CRITERIA — ALL must be met to proceed:

- T1.1 passes: HTTP 200 with ready=true
- T1.2 passes: At least 2 containers discovered automatically
- T1.4 passes: HTTP 204 on /ingest
- T1.5 passes: execution_time_ms < 200ms AND success=true
- T1.7 passes: fallback_triggered=true when < 4 samples (safety gate working)
- T1.9 passes: Memory < 80MB after 5 minutes
- ALL 10 tests must pass. Even 1 failure blocks Phase 2.

■ AI AGENT PROMPT — PHASE 1 — EXECUTION PROMPT

Execute Phase 1 of the ABSIA testing plan exactly as specified. For each test T1.1 through T1.10:

1. Run the exact curl command or docker command specified.
2. Capture the COMPLETE raw response (full JSON, not truncated).
3. Record: actual value observed vs expected value.
4. Mark each test PASS or FAIL based ONLY on the expected result criteria.
5. Save all raw responses to /evidence/phase1/T{N}.json

After all 10 tests, output a Phase 1 Summary table with PASS/FAIL for each test.

If ALL pass, output: 'PHASE 1 COMPLETE — Proceeding to Phase 2'

If ANY fail, output: 'PHASE 1 BLOCKED — Running Phase 1 Failure Diagnosis'

■ AI AGENT PROMPT — PHASE 1 — FAILURE DIAGNOSIS PROMPT

One or more Phase 1 tests have failed. DO NOT proceed to Phase 2. Instead:

1. List every failed test by ID and the exact error or unexpected value observed.
2. For each failure, identify the MOST LIKELY cause from these categories:
 - a) Docker socket permission issue (common on Linux — check /var/run/docker.sock perms)
 - b) Port conflict — is 8080 already in use? Run: ss -tlnp | grep 8080
 - c) Container not starting — check: docker logs absia
 - d) Not enough samples yet — wait 90 seconds and re-run the specific test
 - e) Image pull issue — run: docker pull poonia98/absia:latest explicitly
3. For each identified cause, state the EXACT fix command.
4. Apply ONLY fixes that do not modify ABSIA source code.
5. Re-run ONLY the failed tests after applying fixes.
6. Output updated PASS/FAIL status. If all now pass: 'PHASE 1 RECOVERED — Proceeding.'

If failures persist after 2 diagnosis cycles, STOP and report to human operator.



PHASE 2 — Performance Benchmarking

Measure latency, throughput, and resource usage under controlled load

Objective: Generate quantitative benchmark numbers that go directly into the buyer-facing report. Every number here must be real — measured, not estimated.

2.1 Scale Test — Latency vs Node Count

Inject N nodes via /ingest (4+ samples each) and measure /analyze latency. Run for N = 5, 10, 20, 30, 50 nodes.

```
# Script to inject N nodes and measure analyze latency
for N in 5 10 20 30 50; do
# Inject N nodes with 5 samples each
for i in $(seq 1 $N); do
for s in $(seq 1 5); do
curl -s -X POST http://localhost:8080/ingest \
-H 'Content-Type: application/json' \
-d '{"node_id":"'svc-$i'", "arrival_rate":$((RANDOM%400+100)), "service_rate":$((RANDOM%300+150))
, "queue_length":$((RANDOM%50))}' > /dev/null
done
done
sleep 2
# Measure analyze latency (10 samples, take median)
for run in $(seq 1 10); do
curl -s -X POST http://localhost:8080/analyze \
-H 'Content-Type: application/json' \
-d '{"node_id":"'svc-1'", "arrival_rate":340, "service_rate":200, "queue_length":42}' \
| python3 -c "import sys,json; d=json.load(sys.stdin); print(d.get('execution_time_ms',0))"
done
echo "---N=$N---"
done
```

Target Benchmark Values (buyer-facing table):

Node Count	Analyze Latency Target	Memory Target	Safety Gate Accuracy
5 nodes	< 50ms	< 50MB	100%
10 nodes	< 80ms	< 55MB	100%
20 nodes	< 150ms	< 65MB	100%
30 nodes	< 250ms	< 75MB	100%
50 nodes	< 500ms	< 90MB	> 95%

2.2 Throughput Test — Concurrent Requests

```
# Install hey (HTTP load tester)
go install github.com/rakyll/hey@latest

# 50 concurrent requests, 200 total to /analyze
hey -n 200 -c 50 -m POST \
-H 'Content-Type: application/json' \
-d '{"node_id":"'svc-1'", "arrival_rate":340, "service_rate":200, "queue_length":42}' \
http://localhost:8080/analyze

# Record: requests/sec, p50, p95, p99 latency, error rate
```

2.3 Rate Limiter Verification

```
# Confirm rate limiter fires at > 10 RPS (default ABSIA_RATE_LIMIT_RPS)
hey -n 100 -c 100 -q 50 -m POST \
-H 'Content-Type: application/json' \
-d '{"node_id":"svc-1","arrival_rate":100,"service_rate":90,"queue_length":5}' \
http://localhost:8080/analyze
# Expect: some 429 responses — confirms rate limiter is active
```

■ PHASE PASS CRITERIA — ALL must be met to proceed:

- 5-node analyze latency measured and ≤ 50 ms (actual value recorded)
- 20-node analyze latency measured and ≤ 150 ms (actual value recorded)
- 50-node analyze latency measured (any value — just record it)
- Throughput test: 0% error rate at 50 concurrent requests
- Rate limiter test: HTTP 429 received when sending > 10 RPS
- Memory never exceeds 90MB during any test (docker stats recorded)
- ALL actual measured values saved to `/evidence/phase2/benchmarks.json`

■ AI AGENT PROMPT — PHASE 2 — EXECUTION PROMPT

Execute Phase 2 performance benchmarking exactly as specified.

For the scale test: run the inject-and-measure script for EACH value of N (5,10,20,30,50). For each N, collect 10 latency measurements and compute: min, max, median, p95.

For the throughput test: run hey with the exact parameters given. Record the full output.

For the rate limiter test: confirm 429 responses appear in hey output.

Throughout all tests, run `'docker stats absia --no-stream'` every 30 seconds and record MEM.

Save all raw numbers to `/evidence/phase2/`. Build a summary table with actual vs target.

Output: 'PHASE 2 COMPLETE' only if all pass criteria are met with real measured values.

■ AI AGENT PROMPT — PHASE 2 — FAILURE DIAGNOSIS PROMPT

A Phase 2 benchmark has failed or shows unexpected values. DO NOT proceed. Diagnose:

1. LATENCY TOO HIGH: Check if `/ingest` data from Phase 1 is still in memory. If not, re-inject 5+ samples per node before re-running latency tests. Also check: is Docker Desktop on Mac/Windows? That adds 20-50ms vs Linux host.
2. MEMORY TOO HIGH: Run `'docker exec absia cat /proc/self/status | grep VmRSS'`. If $>$ target, check if temporalMemory global lock is causing accumulation — this is a known bottleneck. Document it honestly in the evidence.
3. ZERO THROUGHPUT / CONNECTION REFUSED: Verify ABSIA is still running: `docker ps`.
4. NO 429 FROM RATE LIMITER: Check if `ABSIA_RATE_LIMIT_RPS` env var is set. Default is 10. If hey is not hitting fast enough, increase `-q` parameter.
5. For any failure: document the actual observed value, not the target. Honest numbers are more valuable than missing numbers.



PHASE 3 — External Real-World Stack Testing

Google Online Boutique (11 services) + Weaveworks Sock Shop (14 services)

Objective: Run ABSIA against production-grade open-source microservice architectures — the same stacks used by Google, Weaveworks, and CNCF for chaos engineering demos. This is the 'real-world evidence' buyers need.

3.1 Google Online Boutique (11 Services)

Online Boutique is Google's reference microservices demo — Go, Python, Node.js, Java, C# services communicating over gRPC. It's what Google uses internally to demo Kubernetes and observability tools.

```
# Clone Google's reference stack
git clone https://github.com/GoogleCloudPlatform/microservices-demo
cd microservices-demo

# IMPORTANT: Use docker-compose version (not k8s) for local testing
# Add ABSIA to their compose file:
cat >> docker-compose.yml <<'EOF'
absia:
  image: poonia98/absia:latest
  ports: ['8080:8080']
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  depends_on: [frontend]
EOF

docker compose up -d
sleep 90 # Wait for all 11 services + ABSIA to stabilize

# Verify ABSIA discovered all 11 services
curl -s http://localhost:8080/nodes | python3 -c \
"import sys,json; d=json.load(sys.stdin); print(f'Discovered: {d[\"node_count\"]} nodes')"
```

3.2 Chaos Injection on Online Boutique

Inject known faults and verify ABSIA identifies the correct root cause. Each scenario must be documented as: fault injected → ABSIA response → correct/incorrect.

Scenario	Fault Type	Injection Method	Expected ABSIA Response
S3.1	CPU Spike	stress-ng on checkout service	root_cause=checkoutservice, confidence > 0.65
S3.2	Network Latency	Pumba adds 300ms to payment service	root_cause=paymentservice OR cascades from it
S3.3	Memory Pressure	stress-ng --vm on recommendation service	status=overloaded for recommendation node
S3.4	Cascade Failure	Kill cart service (upstream dependency stress)	causal chain shows cart as origin
S3.5	Recovery	Remove all stressors, wait 3 min	All nodes return to status=healthy or watch


```
# Scenario S3.1 – CPU Spike on checkout service
docker exec microservices-demo-checkoutservice-1 \
sh -c 'apt-get install -y stress-ng 2>/dev/null; stress-ng --cpu 2 --timeout 120s &'

sleep 75 # Wait for 5 samples (15s * 5)

# Query ABSIA for root cause
curl -s -X POST http://localhost:8080/analyze \
-H 'Content-Type: application/json' \
-d '{"node_id":"checkoutservice","arrival_rate":200,"service_rate":80,"queue_length":15}' \
| python3 -m json.tool

# Save full response to evidence
# Compare root_cause field with expected: checkoutservice

# Scenario S3.2 – Network Latency via Pumba
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock gaiaadm/pumba \
netem --duration 3m --tc-image ghcr.io/alexei-led/pumba-tc \
delay --time 300 microservices-demo-paymentservice-1
```

3.3 Weaveworks Sock Shop (14 Services)

```
# Sock Shop – the CNCF standard chaos engineering benchmark
git clone https://github.com/microservices-demo/microservices-demo
cd microservices-demo

# Add ABSIA
cat >> deploy/docker-compose/docker-compose.yml <<'EOF'
absia:
  image: poonia98/absia:latest
  ports: ['8081:8080']
  volumes:
  - /var/run/docker.sock:/var/run/docker.sock
EOF

docker compose -f deploy/docker-compose/docker-compose.yml up -d
sleep 120 # 14 services take longer to stabilize

# Expected: ABSIA discovers 14+ nodes
curl -s http://localhost:8081/nodes | python3 -c \
"import sys,json; d=json.load(sys.stdin); print(f'Sock Shop nodes: {d[\"node_count\"]}')"

```

■ PHASE PASS CRITERIA — ALL must be met to proceed:

- Online Boutique: ABSIA discovers ≥ 10 of 11 services automatically (no manual config)
- S3.1 (CPU Spike): ABSIA returns `success=true` with `confidence_score > 0.60`
- S3.2 (Network Latency): ABSIA detects load change in the affected service within 2 minutes
- S3.5 (Recovery): After removing stressors, all nodes return to healthy/watch within 4 minutes
- Sock Shop: ABSIA discovers ≥ 12 of 14 services automatically
- Zero crashes or panics in 'docker logs absia' during entire Phase 3
- All 5 chaos scenario responses saved to `/evidence/phase3/S{N}.json`

■ AI AGENT PROMPT — PHASE 3 — EXECUTION PROMPT

Execute Phase 3 external stack testing. Follow this exact sequence:

STEP 1: Clone and start Google Online Boutique with ABSIA attached (as shown above).

STEP 2: Wait 90 seconds. Run GET /nodes. Record node_count and list all node_ids found.

STEP 3: Execute chaos scenarios S3.1 through S3.5 in order. For each:

- Document: timestamp of fault injection
- Document: full /analyze response (raw JSON) 75 seconds after injection
- Document: whether root_cause matches expected (YES/NO/PARTIAL)
- Save response to /evidence/phase3/S{N}_boutique.json

STEP 4: Tear down Online Boutique. Start Sock Shop with ABSIA on port 8081.

STEP 5: Wait 120 seconds. Run GET /nodes on port 8081. Record node_count.

STEP 6: Repeat S3.1 (CPU spike) on Sock Shop's orders service. Record ABSIA response.

STEP 7: Output Phase 3 Summary: discovery counts, scenario pass/fail, any anomalies.

If ABSIA gives wrong root_cause: document it honestly. Wrong answers are evidence too.

■ AI AGENT PROMPT — PHASE 3 — FAILURE DIAGNOSIS PROMPT

Phase 3 issues and fixes:

ISSUE: stress-ng not available in container → Use 'yes > /dev/null &' for CPU stress instead.

ISSUE: Pumba fails → Use 'tc qdisc add dev eth0 root netem delay 300ms' inside container.

ISSUE: ABSIA node_count too low → Check if containers are on same Docker network. ABSIA reads Docker socket — all containers on any network should be visible.

ISSUE: confidence_score always LOW → Check if 4+ samples collected (need 60+ seconds). Query /nodes and verify sample_count >= 4 for target nodes.

ISSUE: root_cause is wrong → DO NOT consider this a failure. Document the incorrect result alongside the correct fault. This is real-world data — imperfect results are honest.

ISSUE: Sock Shop services fail to start → Memory intensive. Ensure host has >= 4GB RAM free.

ISSUE: Docker logs show panics → STOP. This is a real bug. Record full panic trace.

P4

PHASE 4 — Before vs After Comparison

Same failure, same stack — with Prometheus+Grafana vs with ABSIA

Objective: Document the same failure scenario handled by (a) standard Prometheus+Grafana stack and (b) ABSIA. This becomes the 'why ABSIA' slide in your buyer pitch. The comparison must be factual — capture what both tools actually show, not what you wish they showed.

4.1 Setup Prometheus + Grafana Baseline

```
# Add Prometheus + cAdvisor + Grafana to Online Boutique
cat >> docker-compose.yml <<'EOF'
prometheus:
  image: prom/prometheus:latest
  ports: ['9090:9090']
  volumes: ['./prometheus.yml:/etc/prometheus/prometheus.yml']
cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  ports: ['8081:8080']
grafana:
  image: grafana/grafana:latest
  ports: ['3000:3000']
EOF

# prometheus.yml
cat > prometheus.yml <<'EOF'
scrape_configs:
  - job_name: cadvisor
  static_configs:
    - targets: ['cadvisor:8080']
  - job_name: absia
  static_configs:
    - targets: ['absia:8080']
EOF
```

4.2 The Comparison Scenario — Checkout CPU Spike

Inject CPU stress on checkout service. Start a timer. Measure: how long until each tool provides an actionable root cause. Document exactly what each tool shows.

Dimension	Prometheus + Grafana	ABSIA
Discovery	Manual config in prometheus.yml required	Zero config — auto-discovers via Docker socket
Time to First Alert	~2-5 min (scrape interval + alert rule evaluation)	~75 seconds (4 samples x 15s)
What It Shows	CPU % graph spike. Multiple alerts firing.	/analyze: root_cause=checkoutservice, confidence=0.8x
Root Cause	Engineer must correlate multiple graphs manually	Explicit root_cause field with ranked evidence
False Positives	Alerts fire for correlated services too	Safety gate returns UNKNOWN if not confident

Dimension	Prometheus + Grafana	ABSIA
Action Taken	SRE looks at dashboard, decides what to scale	/act returns: scale_out, target=checkoutservice, priority=1
Setup Time	30+ min (prometheus.yml, alert rules, dashboards)	1 command: docker run poonia98/absia

4.3 Evidence to Capture for This Phase

- Screenshot of Grafana dashboard during CPU spike — show all alerts firing
- Raw Prometheus query result: rate(container_cpu_usage_seconds_total[1m]) for each container
- Screenshot of ABSIA dashboard during same CPU spike
- Raw JSON from /analyze during same CPU spike (full response)
- Timer: minutes:seconds from fault injection to actionable information in each tool
- Screenshot of /nodes showing all services with load status
- Output of 'docker logs absia' — confirm no errors during comparison

■ PHASE PASS CRITERIA — ALL must be met to proceed:

- Prometheus + Grafana baseline is running and scraping containers (Grafana shows data)
- Same CPU spike scenario run for BOTH tools, results captured for BOTH
- Time-to-actionable-information measured for both tools (stopwatch, not estimated)
- At least 3 screenshots per tool captured and saved to /evidence/phase4/
- ABSIA /analyze response during fault has success=true (not necessarily correct root cause — just successful response)
- Comparison table filled with ACTUAL observed values for both tools

■ AI AGENT PROMPT — PHASE 4 — EXECUTION PROMPT

Execute Phase 4 before/after comparison. This phase requires timing with a stopwatch.

STEP 1: Start Online Boutique WITH Prometheus+Grafana+cAdvisor+ABSIA all running together.

STEP 2: Wait for Prometheus to have 5+ minutes of baseline data. Confirm Grafana shows graphs.

STEP 3: START TIMER. Inject CPU stress on checkoutservice.

STEP 4: Watch BOTH Grafana and ABSIA dashboard simultaneously.

- Record timestamp when Grafana shows first alert.

- Record timestamp when ABSIA /analyze returns root_cause with success=true.

STEP 5: STOP TIMER. Record both timestamps.

STEP 6: Take screenshots of BOTH tools at the moment they show the issue.

STEP 7: Run 'curl -s http://localhost:9090/api/v1/query?query=rate(container_cpu_usage_seconds_total[1m])' — save output.

STEP 8: Run 'curl -s -X POST http://localhost:8080/analyze ...' — save full JSON response.

STEP 9: Fill the comparison table with actual measured values. No estimates.

Output: 'PHASE 4 COMPLETE' with the time-to-actionable numbers for both tools.

P5

PHASE 5 — AIOps Ground Truth Validation

Test against labeled datasets with KNOWN root causes — the strongest evidence

Objective: Validate ABSIA's causal inference accuracy against publicly available datasets where the correct root cause is already labeled. This is the most credible form of evidence because ground truth is independently established — you didn't make it up.

5.1 Dataset: Alibaba AIOps Challenge Traces

The AIOps Challenge datasets contain real microservice traces from Alibaba's production systems with labeled anomalies. Each entry has: metric values + ground truth root cause node.

```
# Download AIOps 2020 dataset (publicly available)
wget https://github.com/NetManAIOps/AIOps-Challenge-2020-Data/archive/refs/heads/main.zip
unzip main.zip

# The dataset has CSVs with columns:
# timestamp, service_name, cpu_usage, mem_usage, response_time, error_rate, root_cause_label

# Conversion script — translate dataset format to ABSIA /ingest format
python3 <<'EOF'
import csv, requests, time
BASE = 'http://localhost:8080'

with open('dataset/train.csv') as f:
    rows = list(csv.DictReader(f))

# Group by service, inject 5 samples each
services = {}
for row in rows[:500]: # First 500 rows
    svc = row['service_name']
    if svc not in services: services[svc] = []
    services[svc].append(row)

for svc, data in services.items():
    for row in data[:5]: # 5 samples per service
        cpu = float(row.get('cpu_usage', 0.5))
        # Map CPU to arrival/service rates
        arrival = cpu * 500
        service = max(100, (1 - cpu) * 400)
        payload = {'node_id': svc, 'arrival_rate': arrival,
                  'service_rate': service, 'queue_length': arrival/service * 10}
        requests.post(f'{BASE}/ingest', json=payload)
        time.sleep(0.1)

# Now analyze each known-anomaly service
anomalies = [r for r in rows if r.get('is_anomaly') == '1']
for a in anomalies[:10]:
    resp = requests.post(f'{BASE}/analyze',
                        json={'node_id': a['service_name'],
                              'arrival_rate': float(a.get('cpu_usage', 0.5))*500,
                              'service_rate': 200, 'queue_length': 20})
    result = resp.json()
    correct = a['root_cause_label']
    predicted = result.get('root_cause', 'UNKNOWN')
    match = 'CORRECT' if predicted == correct else 'WRONG'
    print(f'{match}: predicted={predicted}, actual={correct},
          confidence={result.get("confidence_score", 0):.3f}')
EOF
```

5.2 Accuracy Reporting Format

Record results honestly. Even 60% accuracy is remarkable for a causal inference tool — most correlation tools give 0% because they don't identify root causes at all.

Metric	Result
Exact Match (predicted == labeled root cause)	X / N tested
Top-3 Match (correct root cause in top 3 candidates)	X / N tested
Safety Gate Triggered (UNKNOWN returned — no false positive)	X / N tested
Wrong Root Cause (confident but incorrect)	X / N tested — MOST IMPORTANT to document

- PHASE PASS CRITERIA — ALL must be met to proceed:
- At least 10 anomaly scenarios from the dataset tested against ABSIA
 - Each test result recorded: predicted root_cause vs labeled root_cause vs CORRECT/WRONG
 - Safety gate behavior documented: how often does ABSIA say UNKNOWN vs give an answer
 - Final accuracy numbers calculated and saved to /evidence/phase5/accuracy_report.json
 - NO requirement on accuracy percentage — honest results are the pass criterion

- AI AGENT PROMPT — PHASE 5 — EXECUTION PROMPT
- Execute Phase 5 ground truth validation.
- STEP 1: Download the AIOps 2020 dataset. If unavailable, use this alternative: `git clone https://github.com/NetManAIOps/GAIA-DataSet` (contains microservice traces).
- STEP 2: Run the conversion script to inject dataset metrics into ABSIA via `/ingest`.
- STEP 3: For each labeled anomaly in the dataset, call `/analyze` and compare predicted root_cause with the labeled root_cause. Record CORRECT, WRONG, or UNKNOWN for each.
- STEP 4: Calculate: `exact_match_rate = correct_count / total_tested`.
- STEP 5: Build the accuracy table with actual numbers.
- STEP 6: Write a 3-sentence honest summary: what ABSIA got right, what it got wrong, and what the safety gate prevented (UNKNOWN responses that avoided false positives).
- This phase is informational — there is no pass/fail on accuracy percentage.
- Save all results to `/evidence/phase5/`.



PHASE 6 — Evidence Compilation & Buyer Report

Assemble all evidence into a professional benchmark document

Objective: Compile all evidence collected in Phases 1-5 into a single buyer-ready benchmark report. This document becomes the primary artifact shared with potential buyers.

6.1 Required Evidence Checklist

Location	Contents Required
/evidence/phase1/	T1.1 through T1.10 raw API responses (10 JSON files)
/evidence/phase2/	benchmarks.json — latency table by node count, memory readings, hey output
/evidence/phase3/	S3.1-S3.5 chaos scenario responses (JSON), discovery counts for both stacks
/evidence/phase4/	Screenshots Grafana, Screenshots ABSIA, time-to-RCA numbers for both
/evidence/phase5/	accuracy_report.json — predicted vs actual for 10+ anomaly scenarios
/evidence/summary/	docker images output, docker stats steady-state, docker logs (no panics)

6.2 Buyer Report Structure

1. Executive Summary — What ABSIA does, who it's for, price
2. Technical Architecture — 5-phase pipeline, zero dependencies, Docker-native
3. Benchmark Results — Latency table (Phase 2), memory footprint, image size
4. Real-World Validation — Online Boutique results (Phase 3), chaos scenarios
5. Before vs After — Prometheus+Grafana vs ABSIA comparison (Phase 4)
6. Accuracy Results — AIOps dataset validation (Phase 5) — honest numbers
7. Safety Gate Proof — Evidence that ABSIA never gives false confident recommendations
8. Getting Started — One docker run command to reproduce every result

■ AI AGENT PROMPT — PHASE 6 — EXECUTION PROMPT

Compile the buyer report using all evidence collected in /evidence/.

STEP 1: Verify all required files exist in each /evidence/phaseN/ directory. If any are missing, identify which phase they come from and re-run that specific test.

STEP 2: Build the benchmark table from /evidence/phase2/benchmarks.json — format as: N nodes | median latency | p95 latency | memory | status.

STEP 3: Build the chaos scenario table from /evidence/phase3/ — format as: scenario | fault injected | ABSIA root_cause returned | correct? | confidence.

STEP 4: Build the before/after table from /evidence/phase4/ — actual measured times for Prometheus vs ABSIA (no estimates).

STEP 5: Extract accuracy numbers from /evidence/phase5/accuracy_report.json.

STEP 6: Write the executive summary in 3 paragraphs: what it is, what evidence shows, who should buy it and why.

STEP 7: Output the complete report as ABSIA_Benchmark_Report.md

The report must contain ZERO estimated or assumed numbers — only evidence-backed values.

QUICK REFERENCE — PHASE GATE SUMMARY

Phase	Name	What Must Exist	Hard Gate Criteria	Then
P0	Pre-Analysis	Repo cloned, all files read, dependency graph built	Readiness Score >= 60/100	Begin Phase 1
P1	Local Baseline	All 10 API tests pass with correct responses	T1.7 safety gate MUST pass	Begin Phase 2
P2	Benchmarks	Latency/memory measured at 5 node counts	5-node latency <= 50ms, 0% error rate at 50 concurrent	Begin Phase 3
P3	External Stacks	10+ nodes discovered, 3+ chaos scenarios run	Zero panics in ABSIA logs, discovery >= 10 nodes	Begin Phase 4
P4	Before/After	Same failure in both Prometheus and ABSIA	Time-to-RCA measured for BOTH tools	Begin Phase 5
P5	Ground Truth	10+ AIOps scenarios tested	All results recorded (no accuracy target)	Begin Phase 6
P6	Buyer Report	All evidence compiled, report written	Zero missing evidence files	DONE — Ship to buyers

This document is the single source of truth for ABSIA real-world testing. The AI agent executing this plan must treat every prompt box as an exact instruction, every pass criterion as a binary gate, and every evidence requirement as mandatory. Honest results — including failures — make the buyer report credible.